Control.Monad.Base: https://hackage.haskell.org/package/transformers-base
Control.Monad.Trans.Control: https://hackage.haskell.org/package/monad-control
Demistifying MonadBaseControl: https://lexi-lambda.github.io/blog/2019/09/07/demystifying-monadbasecontrol/

Control.Monad.Base

```
-- Control.Monad.Base
class (Applicative b, Applicative m, Monad b, Monad m)
=> MonadBase b m | m -> b where
  liftBase :: b a -> m a
```

MonadTransControl

```
-- Control.Monad.Trans.Control
class MonadTrans t => MonadTransControl t where
  type StT t a :: Type
                              -- StT t a = state of (t m
a)
  liftWith :: Monad m => (Run t -> m a) -> t m a
  restoreT :: m (StT t a) -> t m a

type Run t = forall m a. t m a -> m (StT t a)
type RunDefault t t' = forall n b. Monad n => t n b -> n
(StT t' b)

-- MonadTransControl implementation helpers
defaultLiftWith :: (Monad m, MonadTransControl n) =>
  (forall b. n m b -> t m b) -> (forall o b. t o b -> n
o b) -> (RunDefault t n -> m a) -> t m a
defaultRestoreT :: (Monad m, MonadTransControl n) =>
  (n m a -> t m a) -> m (StT n a) -> t m a
type RunDefault2 t n n' = forall m b. (Monad m, Monad
(n' m)) => t m b -> m (StT n' (StT n b))
defaultLiftWith2 :: (Monad m, Monad (n' m),
MonadTransControl n, MonadTransControl n') =>
  (forall b. n (n' m) b -> t m b) -> (forall o b. t o b
-> n (n' o) b) -> (RunDefault2 t n n' -> m a) -> t m a
defaultRestoreT2 :: (Monad m, Monad (n' m),
MonadTransControl n, MonadTransControl n') =>
  (n (n' m) a -> t m a) -> m (StT n' (StT n a)) -> t m a


controlT :: (MonadTransControl t, Monad (t m), Monad m)
=>                    -- controlT (\run -> run (x :: t m
a))
  (Run t -> m (StT t a)) -> t m a
captureT :: (MonadTransControl t, Monad (t m), Monad m)
=> t m (StT t ())
liftThrough :: (MonadTransControl t, Monad (t m), Monad
m) =>
  (m (StT t a) -> m (StT t b)) -> t m a -> t m b
```

MonadBaseControl

```
-- Control.Monad.Trans.Control
class MonadBase b m => MonadBaseControl b m | m -> b
where
  type StM m a :: Type
                      -- StM m a = state of (m a)
  lilftBaseWith :: Monad m => (RunInBase m b -> b a) ->
m a
  restoreM :: b (StM m a) -> m a

type RunInBase m b = forall a. m a -> b (StM m a)

type ComposeSt t m a = StM m (StT t a)

-- MonadBaseControl implementation helpers
type RunInBaseDefault t m b = forall a. t m a -> b
(ComposeSt t m a)
defaultLiftBaseWith :: (MonadTransControl t,
MonadBaseControl b m) =>
  (RunInBaseDefault t m b -> b a) -> t m a
defaultRestoreM :: (MonadTransControl t,
MonadBaseControl b m) =>
  ComposeSt t m a -> t m a


control :: MonadBaseControl b m => (RunInBase m b -> b
(StM m a)) -> m a     -- control (\run -> ...)        run
~ m a -> b a
embed :: MonadBaseControl b m => (a -> m c) -> m (a -> b
(StM m c))
embed_ :: MonadBaseControl b m => (a -> m ()) -> m (a ->
b ())
captureM :: MonadBaseControl b m => m (StM m ())
liftBaseOp :: MonadBaseControl b m =>
                      -- liftBaseOp (\f' -> ...) f
f :: a -> m c    f' ~ a -> b c
  ((a -> b (StM m c)) -> b (StM m d)) -> (a -> m c) -> m
d
liftBaseOp_ :: MonadBaseControl b m =>
                      -- lifeBaseOp_ (\x' -> ...) x
x :: m a         x' ~ b a
  (b (StM m a) -> b (StM m c)) -> m a -> m c

liftBaseDiscard :: MonadBaseControl b m =>
  (b () -> b a) -> m () -> m a
liftBaseOpDiscard :: MonadBaseControl b m =>
  ((a -> b ()) -> b c) -> (a -> m ()) -> m c
```

MonadBase
Like **MonadTrans**, but lifts to the **inner most aka base** layer instead of a single layer

```
    Monad l, Monad m, Monad n
    liftBase :: n a -> l (m (n a))
    liftBase :: n a -> m (n a))
    liftBase :: n a -> n a
```

e.g. liftBase with IO base

```
import Control.Monad.Base
import Control.Monad.Trans
import Control.Monad.Trans.Reader
import Control.Monad.Trans.State
import Control.Monad.Trans.Writer

type MyHandler a = (StateT StateEnv (WriterT Log
(ReaderT Context IO)) a)

hello1 :: MyHandler ()
hello1 = liftBase $ putStrLn "Hello with liftBase"
             -- liftBase :: IO a -> MyHandler a

hello2 :: MyHandler ()
hello2 = lift $ lift $ lift $ putStrLn "Hello with
lift"              -- lift . lift . lift :: IO a ->
MyHandler a
```

MonadTransControl
MoandBaseControl
An upgrade of **MonadTrans**.
Provides a way to temporarily **remove** one transformer **layer** in order to pass to a computation, and **restore** the transformer layer after the computation.
Used when e.g. passing an inner layer to a function **input**
e.g how to use use listen with MyApp?
  type MyApp s w = StateT s (Writer w)
  listen :: WriterT w a -> WriterT (a, w) need to **remove** StateT layer -> run listen -> **restore** StateT layer

Controls

```
  liftBaseOp_ :: MonadBaseControl b m
     => (b (StM m a) -> b (StM m c)) -> m a -> m c
```
e.g. liftBaseOp_ (\x' -> ...) x x :: m a x' :: b (StM m a) ~ x' :: b a

```
  liftBaseOp :: MonadBaseControl b m
      => ((a -> b (StM m c)) -> b (StM m d)) -> (a -> m c) -> m d
```
e.g. liftBaseOp (\f' -> ...) f f :: a -> m c f' :: a -> b (StM m c) ~ f' :: a -> b c

```
  control :: MonadBaseControl b
```
base of liftBaseOp/liftBaseOp_
```
    => (RunInBase m b -> b (StM m a)) -> m a
liftBaseOp_ f x = control $ \run -> f (run x)

liftBaseOp f g = control $ \run -> f (run . g)
```

Example: Basic

```
import Control.Monad.Base
import Control.Monad.Trans
import Control.Monad.Trans.Control
import Control.Monad.Trans.Reader
import Control.Monad.Trans.State
import Control.Monad.Trans.Writer
import Data.Function
import System.IO

type MyLog = String
type MyEnv = Bool
type MyState = Int

type MyHandler a = (StateT MyState (WriterT MyLog
(ReaderT MyEnv IO)) a)

myListen :: MyHandler a -> MyHandler (a, MyLog)
                              -- listen :: WriterT w a ->
WriterT (a, w) for MyHandler
myListen h = controlT $ \run -> do
  ((a, st), log) <- listen $ run h
  return ((a, log), st)

myLocal :: (MyEnv -> MyEnv) -> MyHandler a -> MyHandler
a                           -- local :: (r -> r) ->
ReaderT r a -> ReaderT r a for MyHandler
myLocal f h = controlT $ \runSt -> controlT $ \runW ->
local f $ runW $ runSt h

myWithFile :: FilePath -> IOMode -> (Handle -> MyHandler
a) -> MyHandler a         -- writeFile :: FilePath ->
IOMode -> (Handle -> IO a) -> IO a for MyHandler
myWithFile path mode = liftBaseOp $ \lifted -> withFile
path mode lifted

myWithFile2 :: FilePath -> IOMode -> (Handle ->
MyHandler a) -> MyHandler a
myWithFile2 path mode f = control $ \run -> withFile
path mode (run . f)
```

Usage

```
myComputation :: MyHandler ()
myComputation = do
  put 1
               -- state
  lift $ tell "HELLO"
             -- writer
  r <- lift $ lift ask
            -- reader
```

```
  lift $ tell $ "env:" <> show r
  return ()


main :: IO ()
main = do
  _ <- runHandler $ myListen someComputation
             -- myListen

  _ <- runHandler $ myLocal not myComputation
            -- myLocal

  _ <- runHandler $
              -- myWithFile
    myWithFile "/tmp/test.txt" WriteMode $ \handle -> do
      lift $ tell "Writing to file"
      liftBase $ hPutStrLn handle "Hello"

runHandler :: Show a => MyHandler a -> IO (MyLog,
MyState, a)
runHandler handler = do
  let handler' = do
          r <- lift $ lift ask
          a <- handler
          return (a, r)
  result@(((a, _), s), w) <-
    runStateT handler' 0
      & runWriterT
      & flip runReaderT True
  print result
  return (w, s, a)
```

Output

```
(((((),"HELLOenv:True"),True),1),"HELLOenv:True")
         -- myListen

((((),True),1),"HELLOenv:False")
          -- myLocal

((((),True),0),"Writing to file")
         -- myWithFile
```

Examples

```
import Control.Monad.Trans.Control

-- withFile
withFile :: FilePath -> IOMode -> (Handle -> IO a) -> IO
a
```

```
withFileM :: MonadBaseControl IO m => FilePath -> IOMode
-> (Handle -> m a) -> m a
withFileM path mode = liftBaseOp $ \lifted -> withFile
path mode lifted

import Control.Monad.Catch
-- catch
catch :: Exception e => IO a -> (e -> IO a) -> IO a
catchM :: (MonadBaseControl IO m, Exception e)  => m a
-> (e -> m a) -> m a
catchM x h = control $ \run -> catch (run x) (run . h)
```